ARQUITETURA DE SOFTWARE · AULA 09 · APRESENTAÇÃO EM DUPLA

MVC na prática com Express + EJS

Funcionalidade: **empréstimo de livros** numa biblioteca — com regra de negócio, da rota até a tela.

Lucas Roldão Cardoso · **Abner Bergmüller Silva da Silva**

CST Análise e Desenvolvimento de Sistemas · ULBRA Torres

O FRAMEWORK E O MAPA

Onde mora cada parte

Express + EJS

Node.js / JavaScript. Framework web minimalista, usado em sites e APIs. O **EJS** é o motor de template: transforma HTML + dados na página.

MODEL dados + regra de negócio

CONTROLLER recebe, delega, escolhe a View

VIEW só exibe o que recebe

```
biblioteca/
  server.js                          liga tudo e configura a View
  src/
    routes.js                        mapa de rotas
    models/
      biblioteca.js                  MODEL · dados + regra
    controllers/
      livros.controller.js           CONTROLLER
    views/
      livros.ejs                     VIEW
  test/
      biblioteca.test.js             testa o Model sem HTTP
```

Rota → Controller → Model

routes.js

```
router.post("/livros/:id/emprestar", livros.emprestar);
```

models/biblioteca.js

```
emprestar(id, matricula) {
  const livro = this.buscar(id);

  if (livro.emprestadoPara)
    throw new LivroIndisponivel(
      `"${livro.titulo}" já está emprestado.`);

  if (this.emprestimosDo(matricula) >= LIMITE)
    throw new LimiteDeEmprestimos(
      `Matrícula ${matricula}: limite de ${LIMITE}.`);

  livro.emprestadoPara = matricula;
}
```

MODEL a regra: livro livre e máx. 2 por matrícula

controllers/livros.controller.js

```
emprestar(req, res) {
  const id = Number(req.params.id);
  const matricula = (req.body.matricula || "").trim();

  if (!MATRICULA.test(matricula))
    return pagina(res, 400,
      { erro: "Matrícula: 6 dígitos." });

  try {
    biblioteca.emprestar(id, matricula);
    return pagina(res, 200, { ok: "..." });
  } catch (e) {
    return pagina(res, 422,
      { erro: e.message });
  }
}
```

CONTROLLER valida o formato, chama o Model, escolhe a View e o status

O CÓDIGO, DE PONTA A PONTA · 2/2

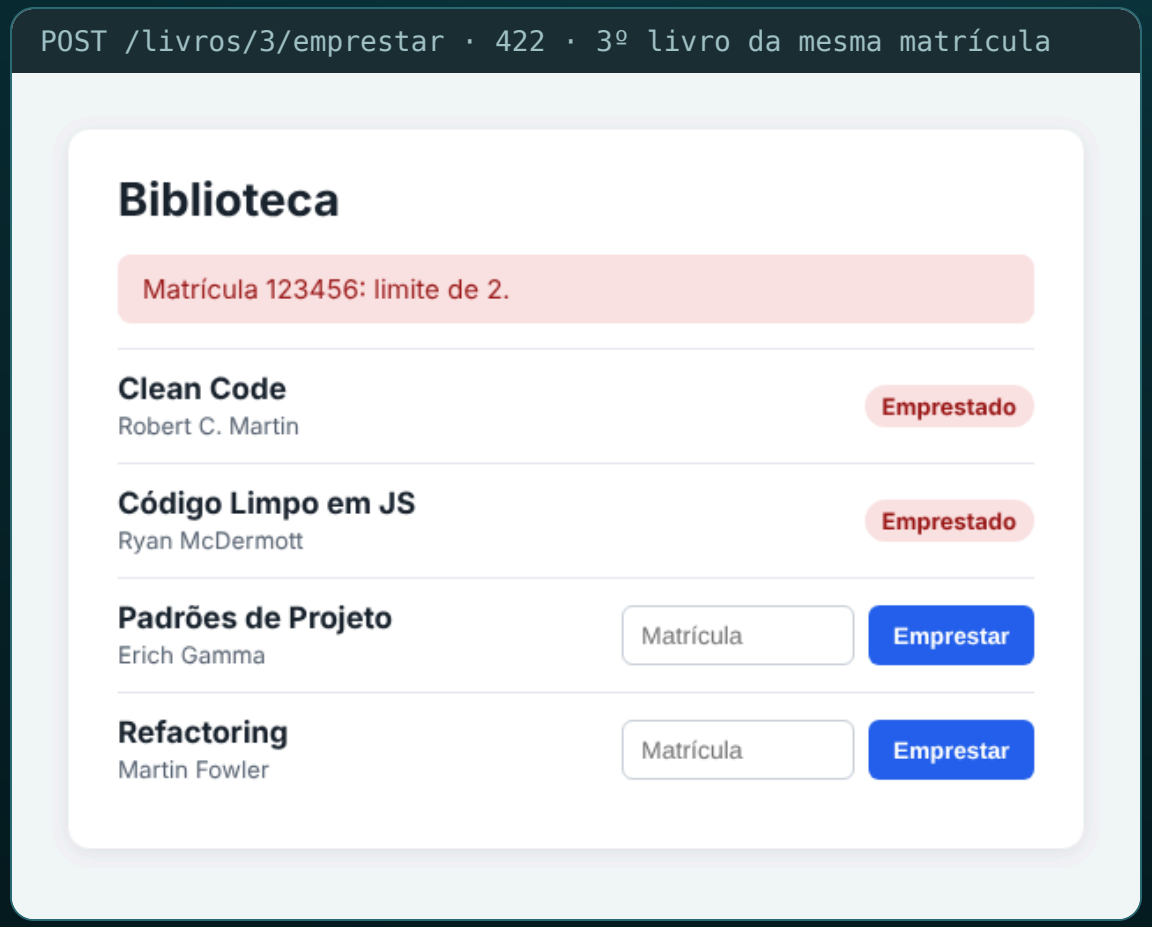
View → a tela que aparece

views/livros.ejs

```
<% if (erro) { %>
  <div class="msg erro"><%= erro %></div>
<% } %>

<% livros.forEach(function (livro) { %>
  <b><%= livro.titulo %></b>
  <% if (livro.disponivel) { %>
    <form method="post" action="...">
      ...
    </form>
  <% } else { %>
    <span class="selo">Emprestado</span>
  <% } %>
<% }) %>
```

POST /livros/3/emprestar · 422 · 3º livro da mesma matrícula



VIEW o único `if` pergunta `livro.disponivel`, calculado no Model

O CONCEITO

Cada parte faz só o que lhe cabe?

Regra × formato

A **regra** mora no Model: `Biblioteca.emprestar()` (livro livre, máx. 2 por matrícula). O **formato** (matrícula com 6 dígitos) é validado no Controller, que devolve 400.

O Controller só traduz?

Sim: traduz a requisição, delega e escolhe a View e o status (200, 400, 422). Ele decide **o que mostrar**, não **o que é permitido** — não tem nenhum `if` de negócio.

A View só exibe?

Sim. Só tem laço e `<%= %>`. O único `if` pergunta `livro.disponivel`; a conta de quem está emprestado fica no Model.

O teste da separação

Passa: `npm test` roda 3 testes da regra (livro já emprestado, limite de 2) **sem subir servidor e sem requisição HTTP**.

UMA DIFERENÇA

Onde o Express se afasta do MVC da aula

Não impõe MVC

O Express só tem rotas e `res.render`. As pastas `models/`, `controllers/` e `views/` são **convenção nossa**; Rails, Laravel e Django já trazem isso pronto.

Model sem ORM

Aqui o Model é uma classe comum e os dados ficam num array em memória. Num sistema real ele acessaria o banco e juntaria **domínio + acesso a dados** (o "MVC × camadas" da aula).

Sem camada de serviço

O Controller chama o Model direto, porque o projeto é pequeno. Num sistema maior surgiria um **service** no meio, como no NestJS do material complementar.

Conclusão: a regra está no Model, o Controller só coordena e a View só exibe. Por isso a mesma regra vale pela tela e pelo teste.